

Under the default configuration all data will be stored in the `./qdrant_storage` directory. This will also be the only directory that both the container and the host machine can see.

Qdrant is now accessible:

- REST API: `localhost:6333`
- Web UI: `localhost:6333/dashboard`
- GRPC API: `localhost:6334`

Step 2: Initialising Qdrant client, creating a collection, and generating embeddings

Now that Qdrant is running and your Python environment is set up, let's start writing some code to interact with it. We'll initialise the Qdrant client, create a collection to store our vectors, and use a pre-trained model to generate vector embeddings from text.

Initialise the Qdrant client: First, open a new Python file (e.g., `qdrant_tutorial.py`) in your project directory and add the codes below. We need to connect to our running Qdrant instance.

```
from qdrant_client import QdrantClient, models
from sentence_transformers import SentenceTransformer
import uuid

# Initialize Qdrant client
# We are connecting to the Qdrant instance running locally on
# port 6333
client = QdrantClient (host="localhost", port=6333)

print("Qdrant client initialized.")
```

The `QdrantClient` is the main interface for interacting with your Qdrant database. We're connecting to `localhost` on port `6333`, which is the default gRPC port for Qdrant in Docker.

Define our data and embedding model: To demonstrate semantic search, we need some text data and a way to convert that text into numerical vectors (embeddings). We'll use the `SentenceTransformer` library, which provides easy access to pre-trained models.

```
# Our sample data
documents = [
    "Qdrant is a vector similarity search engine.",
    "It helps in finding relevant information quickly.",
    "Vector databases are essential for modern AI applications like RAG.",
    "Machine learning models can generate powerful embeddings.",
    "The Qdrant dashboard provides a great overview of collections.",
    "SentenceTransformers can create semantic vector embeddings.",
    "Langchain often integrates with Qdrant for RAG applications.",
```

```
"Understanding embeddings is key to effective vector search."
]
# Initialize a SentenceTransformer model for generating
embeddings
# 'all-MiniLM-L6-v2' is a good balance of performance and size
embedding_model = SentenceTransformer("all-MiniLM-L6-v2")
print("Embedding model loaded.")
```

Here, `documents` is a list of simple text strings that we'll convert into vectors. The `SentenceTransformer("all-MiniLM-L6-v2")` loads a pre-trained model capable of generating high-quality semantic embeddings. This model will convert each sentence into a fixed-size numerical vector where semantically similar sentences will have vectors that are numerically close to each other.

Create a Qdrant collection: Before we can add vectors, we need to create a 'collection' in Qdrant. A collection is like a table in a relational database, but it's designed specifically for storing vectors and their associated metadata. When creating a collection, you must specify the `vector_size` (the dimensionality of your embeddings) and the distance metric (how similarity between vectors is calculated).

```
collection_name = "my_first_collection"
# Get the vector size from our embedding model
# The 'all-MiniLM-L6-v2' model produces 384-dimensional vectors

vector_size = embedding_model.get_sentence_embedding_dimension()

# Check if the collection already exists and delete it if it
# does
# This is useful for re-running the tutorial script
if client.collection_exists(collection_name=collection_name):
    client.delete_collection(collection_name=collection_name)
    print(f"Collection '{collection_name}' deleted (if it
    existed).")
# Create the collection
client.create_collection(
    collection_name=collection_name,
    vectors_config=models.VectorParams(size=vector_size,
```

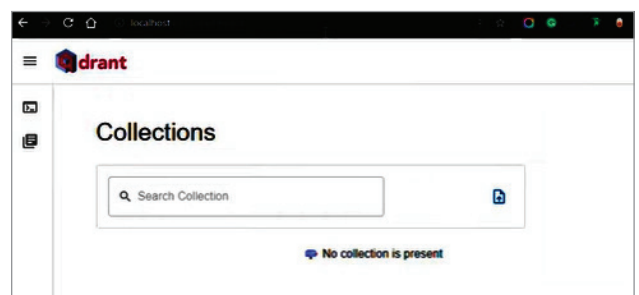


Figure 2: Qdrant user interface